

SORTING

PROBLEM 4

QUESTION:

DS	Session	Sorting	Question Information	Level 1	Challenge 14
Question description APPU needed a laptop, so he went to a neighbouring garage sale. At a sale, there were n laptops. The cost of a laptop with index i is a_i rupees. Some laptops have a negative price, meaning their owners are willing to pay APPU if he buys their broken laptop. APPU is free to purchase whichever laptop he desires. Despite his strength, he can only handle m Laptops at a time, and he has no desire to return to the auction. Please, help APPU find out the maximum sum of money that he can earn. Constraints: $1 \leq T \leq 10$ $1 \leq n, m \leq 100$ $-1000 \leq a_i \leq 1000$ Input Format: First line of the input contains T denoting the number of test cases. Each test case has 2 lines : first line has two spaced integers n m. second line has n integers $[a_0 \dots a_{i-1} \dots a_{n-1}]$. Output Format:					

The maximum sum of money that LALU can earn, given that he can carry at most m Laptops.

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 1 5 3 -6 0 35 -2 4 EXPECTED OUTPUT 8	INPUT (STDIN) 2 6 2 -4 5 0 -1 2 1 7 4 6 1 0 -2 2 -4 3 EXPECTED OUTPUT 5 6

Mandatory Test Cases

Test Case 1	Test Case 2
KEYWORD int MEGA_SALE(int arr[],int no,int k)	KEYWORD void bubble_sort(int arr[],int no)

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 4	TOKEN COUNT 320	NLOC 55

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>

int cmp_increasing(const void* a, const void* b) {
    return (*(const int*)a - *(const int*)b);
}

int main() {
    int batches;
    if (scanf("%d", &batches) != 1) return 0;

    while (batches--) {
        int available, max_picks;
        scanf("%d %d", &available, &max_picks);

        int prices[available];
        for (int i = 0; i < available; i++) scanf("%d", &prices[i]);

        qsort(prices, available, sizeof(int), cmp_increasing);

        int accumulated_earnings = 0;
        for (int i = 0; i < max_picks && i < available; i++) {
            if (prices[i] < 0) {
                accumulated_earnings += (-prices[i]);
            }
        }
        printf("%d\n", accumulated_earnings);
    }
    return 0;
}
```

OUTPUT:

```
2
5 3
-6 0 35 -2 4
4 2
-1 -5 2 3
8
6
```

```
2
6 2
-4 5 0 -1 2 1
7 4
6 1 0 -2 2 -4 3
5
6
```